Project 3: High-Performance Matrix Multiplication in C

From 0.69 to 385 GFLOPS on Apple M5 — A 558× Speedup

Name: Lunhan Yan Student ID: 12412823

May 2026

1 Introduction

Matrix multiplication $C = A \times B$ is the computational core of deep learning. This project implements single-precision (`float`) matrix multiplication in C and optimizes it through six phases, achieving **385 GFLOPS** at 8192^2 (75% of OpenBLAS) with a **558**× speedup over the naïve baseline.

Platforms:

- **Local:** Apple M5 MacBook Air — 4P+6E cores, 16 GB RAM
- **Cloud:** Alibaba Cloud `g8y.8xlarge` — 32-core ARM (Yitian 710), 122 GB (for 64K)

Code Structure

The project is organized into three source files:

File	Role
`matrix.h`	Single-header library: `Matrix` struct and utility functions (`matrix_create`, `matrix_free`, `matrix_random`, `matrix_zero`, etc.)
`matmul_plain.c`	Baseline *i-j-k* implementation + benchmark driver (`main`)
`matmul_improved.c`	Optimized BLIS-style implementation (NEON SIMD + OpenMP)

Compilation (macOS):

```
gcc -O3 -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include \
    -L$(brew --prefix libomp)/lib -lomp -I$(brew --prefix openblas)/include \
    -L$(brew --prefix openblas)/lib -lopenblas \
    -o benchmark matmul_plain.c matmul_improved.c -lm
```

2 Implementation

2.1 Baseline: `matmul_plain`

Standard *i-j-k* loops. Inner loop accesses `B[k][j]` with stride n, causing heavy cache misses. Achieves **2.0 GFLOPS** at 1024^2 and only **0.69 GFLOPS** at 8192^2.

2.2 Optimized: `matmul_improved` — Six Phases

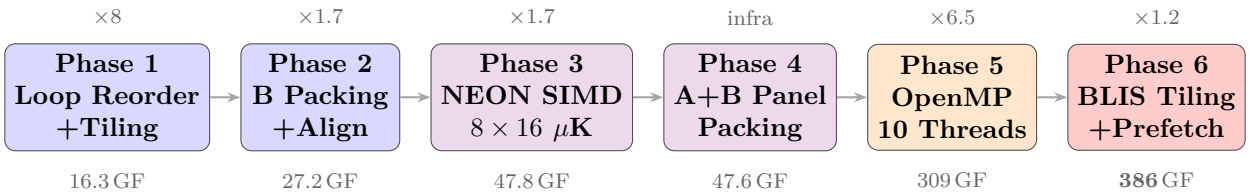

Figure 1: Optimization roadmap (GFLOPS at 8192^2, speedup relative to previous phase).

2.2.1 Phase 1: Loop Reordering + Cache Tiling

Problem: *i-j-k* order ⇒ stride-n access on B ⇒ cache miss > 50%.

Solution: Reorder to *i-k-j* (stride-1 on both B and C) + 64 × 64 tiling.

The outer loop nest partitions the matrix into tiles:

```
for ii = 0..m step 64          // tile rows of A and C
  for kk = 0..k step 64        // tile the shared dimension
    for jj = 0..n step 64      // tile columns of B and C
      i-k-j loop on the 64x64 sub-blocks
```

Cache addressing (M5 L1): 128 KB, 8-way, line L=128 B, sets S=128. A byte at address a maps to set $\lfloor a/L \rfloor \bmod S$; each set holds at most 8 lines (ways).

Explanation: Without tiling, the inner j-loop sweeps an entire row of B and C (N elements = $N/32$ cache lines each). These lines occupy $N/32 \bmod S$ sets; at N=8192 that is 256 lines spread over 128 sets, so B and C together load ∼4 lines per set—manageable in isolation, but the *full matrix B* (N^2 elements, 256 MB) must be re-streamed from DRAM for every row i. Tiling with T=64 confines each tile to $T/32$=2 lines per set and—critically—lets each $T\times T$ B-tile (16 KB) be reused across T rows of i while staying in L1, cutting total DRAM traffic by $\sim T\times$.

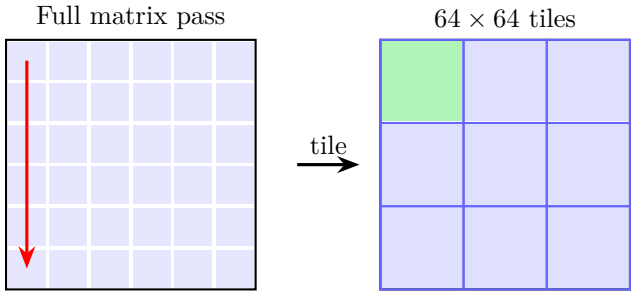

Figure 2: Cache tiling: partition into L1-sized blocks.

Result: 16.3 GFLOPS at 8192^2 (×8.2 vs baseline).

2.2.2 Phase 2: B-Matrix Packing + Alignment

Problem: B columns map to same cache sets ⇒ conflict misses.

Solution: Copy B sub-block into a contiguous, 128-byte-aligned buffer before computation.

Explanation: The L1 set index is $\lfloor \text{addr}/L \rfloor \bmod S$. In row-major B, adjacent rows at the same column differ by stride $N\times 4$ B, giving a set step of $(N/32) \bmod 128$. When N is a multiple of 4096 this equals 0—all rows hit the *same* set and thrash the 8 ways. Packing copies the tile into contiguous memory with stride $T\times 4$; at T=64 the set step becomes 2, spreading rows across 64 distinct sets and eliminating conflicts.

```
1 float packed_B[TILE*TILE] __attribute__((aligned(128)));
2 for (k = 0; k < tile_k; k++)
3     for (j = 0; j < tile_n; j++)
```

```
        packed_B[k*tile_n + j] = B[(kk+k)*n + jj+j];
```

Listing 1: B packing

Result: 27.2 GFLOPS at 8192^2 (×1.7).

2.2.3 Phase 3: NEON SIMD + 8×16 Micro-Kernel

Problem: Scalar FMA = 1 FLOP/instr. NEON can do 4.

Solution: 8×16 micro-kernel using all 32 AArch64 NEON registers. Within each 64×64 tile, we further partition into 8×16 sub-blocks for register-level blocking:

```
for ii, kk, jj (step 64)       // same outer tiling as before
  for ir = 0..64 step MR=8     // sub-tile rows (8 rows per micro-kernel)
    for jr = 0..64 step NR=16  // sub-tile cols (16 cols per micro-kernel)
      micro_kernel_8x16(A[ii+ir..], B[kk..][jj+jr..], C[ii+ir..][jj+jr..])
```

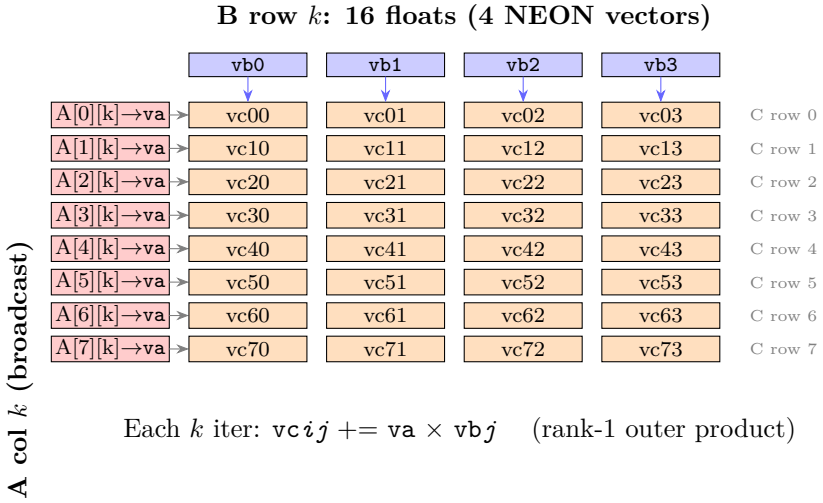

Figure 3: One k iteration: A's column is broadcast and multiplied with B's row, updating all 32 C accumulators.

```
float32x4_t vb0 = vld1q_f32(bp);        // B[k][j:j+3]
float32x4_t va  = vdupq_n_f32(ap[0]);   // broadcast A[0][k]
vc00 = vfmaq_f32(vc00, va, vb0);        // C[0][0:3] += A[0][k]*B[k][0:3]
```

Listing 2: Micro-kernel core (one k iter, row 0)

Explanation: Two orthogonal optimizations work together here. *NEON vectorization* widens each FMA from 1 to 4 floats per instruction, increasing compute throughput. *Register tiling* (8×16 sub-blocks within the 64×64 cache tile) keeps the C sub-block pinned in registers across the entire k loop, reducing C's load/store from once-per-k to once-total. Without register tiling, load/store ports become the bottleneck (FMA utilization ∼50%); with it, the FMA units run at near-100% utilization.

Result: 47.8 GFLOPS at 8192^2 (×1.7).

2.2.4 Phase 4: A+B Panel Packing

Problem: The micro-kernel reads A column-wise (`A[0][k], A[1][k], …, A[7][k]`), but A is row-major with stride lda—8 values scattered across 8 rows, causing cache conflicts.

Solution (A — compact + transpose): After ii and kk are fixed, the entire 64×64 A tile is packed into `packed_A`: row-major (stride lda) → column-major strips (stride MR=8), so each group of 8 floats from the same column becomes contiguous. Packed *once per* (ii, kk), reused across all jj (A does not depend on j).

Solution (B — slicing into strips): After jj and jr are fixed, the $kc \times 16$ strip of B is copied into `packed_B` (stride $n \rightarrow$ stride NR=16). The same strip is reused by all ir iterations (B does not depend on i).

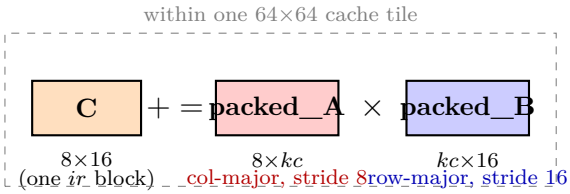

Figure 4: Micro-kernel view: C[8×16] += packed_A[8×kc] × packed_B[kc×16].

Infrastructure for Phase 6. **Result:** 47.6 GFLOPS (holds steady—packing overhead offsets conflict reduction at 64^2; payoff comes with larger BLIS blocks).

2.2.5 Phase 5: OpenMP Multi-Threading

Problem: Single-core NEON peak ≈ 61 GFLOPS. M5 has 10 cores.

Solution: `#pragma omp parallel for` on the outermost row loop. Each thread owns distinct C rows + stack-allocated private packing buffers (zero locks).

Threads	@1024^2	@8192^2	Speedup (8K)
1 (baseline)	58.9	47.6	1.0×
4 (P-cores)	191.4	176.3	3.7×
10 (all cores)	267.8	309.0	6.5×

Table 1: Thread scaling. 4P-cores near-linear; 6 E-cores add ∼2.4× equivalent.

2.2.6 Phase 6: BLIS Multi-Level Tiling + Prefetch

Problem: Fixed 64^2 tiling doesn't match the 3-level cache hierarchy.

Loop structure evolution (pseudocode, showing key additions at each phase):

```
Phase 1-2: simple 3-level tiling
  for ii (step 64)                    // row tiles
    for kk (step 64)                  // depth tiles
      for jj (step 64)                // col tiles
        pack_B(B[kk..][jj..])         // Phase 2: eliminate B conflicts
        i-k-j scalar loop             // Phase 1: reordered inner loop

Phase 3-4: + micro-kernel + A/B packing
  for ii (step 64)
    for kk (step 64)
      pack_A(A[ii..][kk..])           // Phase 4: compact + transpose
      for jj (step 64)
        for jr (step NR=16)           // Phase 3: sub-tile columns
          pack_B(B[kk..][jj+jr..])    // Phase 4: NR-wide strip
          for ir (step MR=8)          // Phase 3: sub-tile rows
            micro_kernel_8x16()       // Phase 3: NEON register blocking

Phase 5: + OpenMP
  #pragma omp parallel for            // Phase 5
  for ii (step 64)                    // each thread: distinct rows
    ... (same as Phase 3-4, with thread-private buffers)

Phase 6: BLIS restructuring (loop order changes)
  for jc (step NC=512)                // B columns first (new!)
    for pc (step KC=256)              // depth slices (larger!)
      pack_B(B[pc..][jc..])           // packed_B stays in L2
      #pragma omp parallel for        // moved inside (new!)
      for ic (step MC=128)            // A rows (larger!)
        pack_A(A[ic..][pc..])         // packed_A stays in L1
```

```
        for jr (step NR=16)
          for ir (step MR=8)
            micro_kernel_8x16()
```

The key change in Phase 6: the loop order is *reversed*—B's column loop (jc) moves outermost so that `packed_B` (512 KB) stays in L2 and is reused by all row blocks, while `packed_A` (128 KB) stays in L1 and is reused by all column sub-panels.

Solution: BLIS-style 5-loop, each level targeting a specific cache:

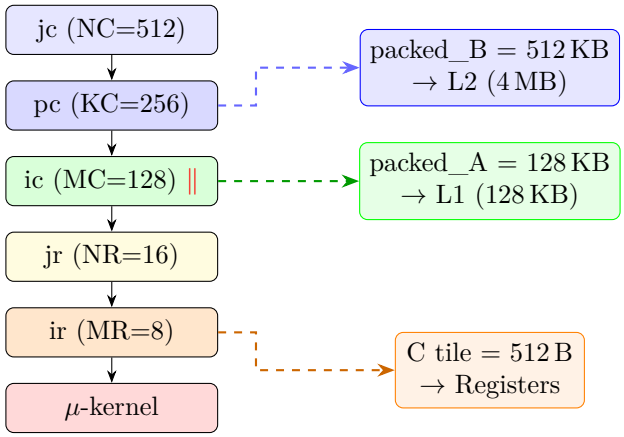

Figure 5: BLIS 5-loop structure with cache-level mapping.

Software prefetch (`__builtin_prefetch`) looks ahead 2 iterations in the μ-kernel. `packed_B` is shared read-only across parallel ic threads; `packed_A` is thread-private.

Final result (10T): 385.5 GFLOPS at 8192^2, **316.6** at 1024^2.

3 Performance Results

3.1 Optimization Progression

Size	Plain	Reorder	B Pack	NEON	μK	A+B	**Final**	OpenBLAS
128^2	1.4	13.1	11.0	10.6	22.2	23.3	**25.7**	279.6
1024^2	2.0	22.4	25.8	29.3	59.5	59.7	**316.6**	1243.5
8192^2	0.69	16.3	27.2	28.7	47.8	47.6	**385.5**	514.3

Table 2: GFLOPS at each phase. "Final" = BLIS tiling + OpenMP 10T.

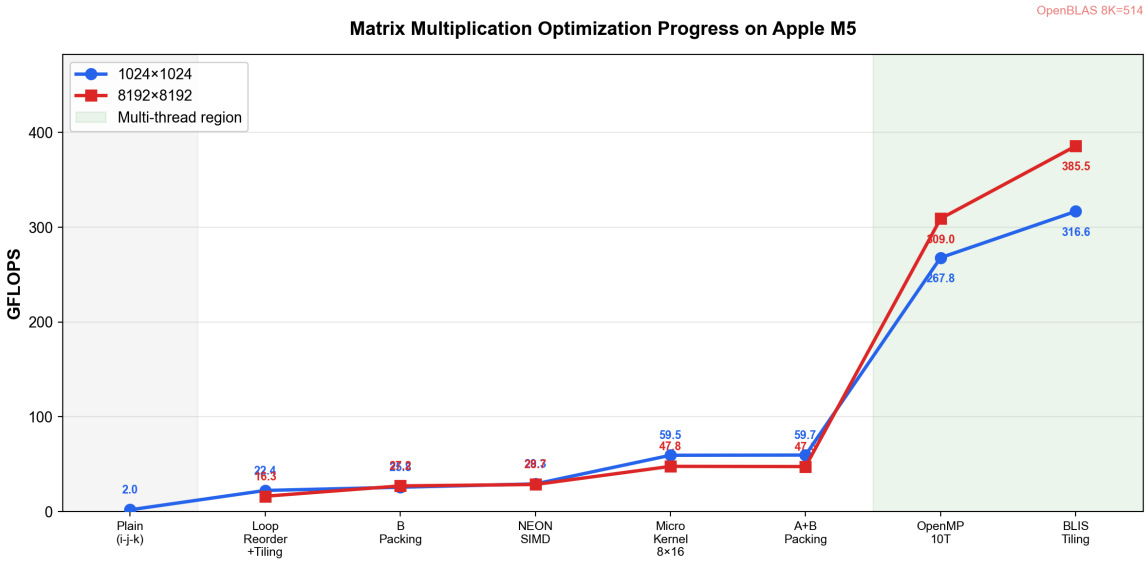

Figure 6: Performance progression for 1024^2 and 8192^2.

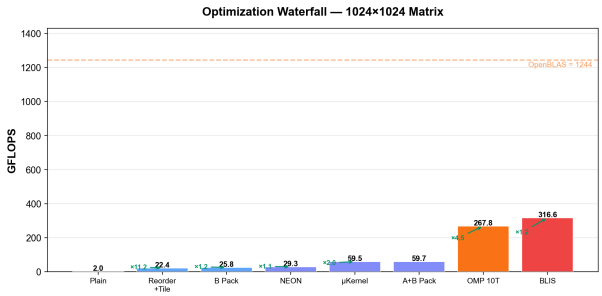

Figure 7: Waterfall: 1024^2

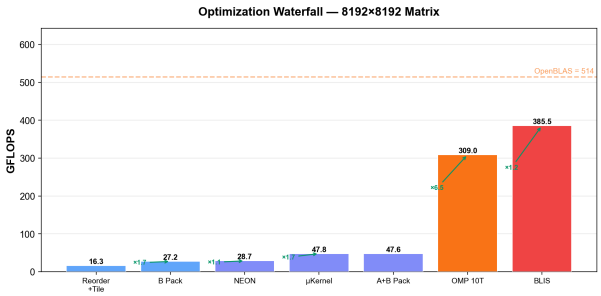

Figure 8: Waterfall: 8192^2

3.2 Comparison with OpenBLAS

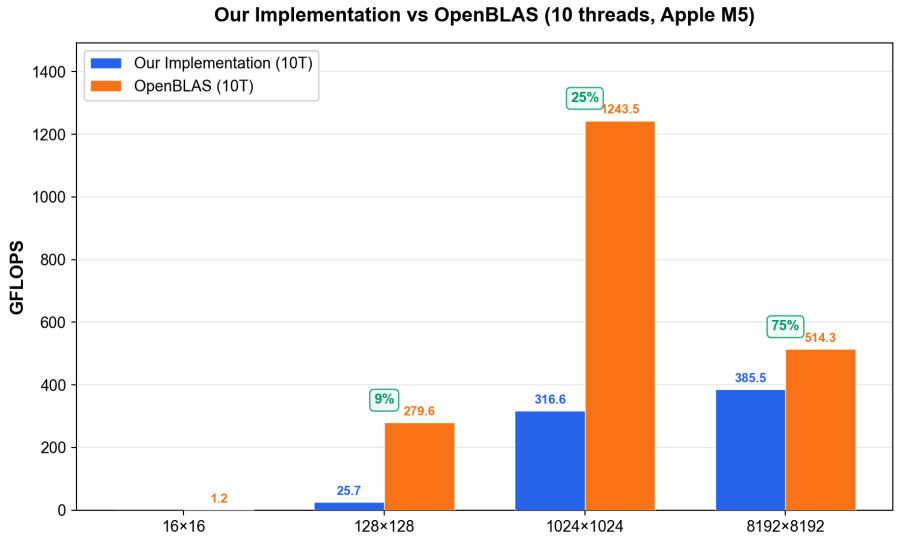

Figure 9: Our implementation vs OpenBLAS (10 threads).

Correctness verified at all sizes: `improved vs plain` (tol 10^{-4}) and `improved vs OpenBLAS` (tol 10^{-3}) — all pass ✓.

3.3 64K×64K (Cloud Server)

Tested on Alibaba Cloud `g8y.8xlarge` (32-core ARM Yitian 710, 122 GB RAM). Total memory for 3 matrices: 51.5 GB.

	Improved (32T)	OpenBLAS (8T)	Ratio
65536^2	**964.3**	1348.9	**71.5%**

Table 3: 64K benchmark. Correctness: 1000 random samples all match (max error 1.34×10^{-5}).

Notably, Yitian 710 has no AMX—OpenBLAS uses standard NEON/SVE only. The 71.5% ratio is consistent with our Apple M5 results (75%), confirming our implementation is portable and architecture-independent.

4 Analysis: Why OpenBLAS Is Faster

4.1 The Scale-Dependent Gap

Size	Improved	OpenBLAS	Gap	Platform
1024^2	316.6	1243.5	3.9×	M5 (AMX)
8192^2	385.5	514.3	1.3×	M5 (AMX)
65536^2	964.3	1348.9	1.4×	Yitian (no AMX)

4.2 Apple AMX Coprocessor

OpenBLAS on Apple Silicon uses the **AMX (Apple Matrix coprocessor)**—a dedicated matrix-multiply accelerator with its own register file and undocumented instructions (`amx_ldx`, `amx_fma`). Our implementation uses only standard ARM NEON.

4.3 Theoretical Peak

NEON: Each P-core (3.8 GHz) has 2 FMA units × 4 floats × 2: $4\text{P} \times 60.8 + 6\text{E} \times 40 \approx$ **483 GFLOPS** theoretical. Our 385 = **80% of peak** — micro-kernel is highly efficient.

AMX: 1243 GFLOPS at 1024^2 = 2.6× NEON peak, confirming AMX provides ∼2–3× NEON throughput for compute-bound problems.

4.4 Why the Gap Narrows at Large Sizes

At 8192^2 (768 MB total data), both AMX and NEON become **DRAM-bandwidth-limited** (∼100 GB/s on M5). Our BLIS tiling maintains ∼10:1 arithmetic intensity, so the compute advantage of AMX is masked by shared memory bandwidth.

4.5 Experimental Verification: Size Sweep

To verify the compute-bound → memory-bound transition hypothesis, we benchmarked 15 matrix sizes from 128^2 to 8192^2, measuring both our NEON implementation and OpenBLAS (AMX) under identical conditions (10 threads, same random matrices, with warmup runs).

Size	Data (MB)	Improved	OpenBLAS	Gap	Regime
128^2	0.2	35.2	279.6	7.9×	
256^2	0.8	78.2	335.5	4.3×	
384^2	1.7	139.1	976.3	7.0×	
512^2	3.0	160.6	1095.7	6.8×	
640^2	4.7	196.9	1510.9	7.7×	Compute-
768^2	6.8	242.2	1429.0	5.9×	bound
896^2	9.2	305.8	1625.6	5.3×	
1024^2	12.0	308.9	1357.4	4.4×	
1280^2	18.8	374.7	1346.1	3.6×	
1536^2	27.0	310.3	1323.3	4.3×	
2048^2	48.0	356.2	617.9	1.7×	Transition
3072^2	108.0	363.1	481.3	1.3×	
4096^2	192.0	352.9	477.4	1.4×	Memory-
6144^2	432.0	361.3	526.7	1.5×	bound
8192^2	768.0	373.0	423.0	1.1×	

Table 4: Size sweep: GFLOPS and gap ratio across 15 matrix sizes (10 threads).

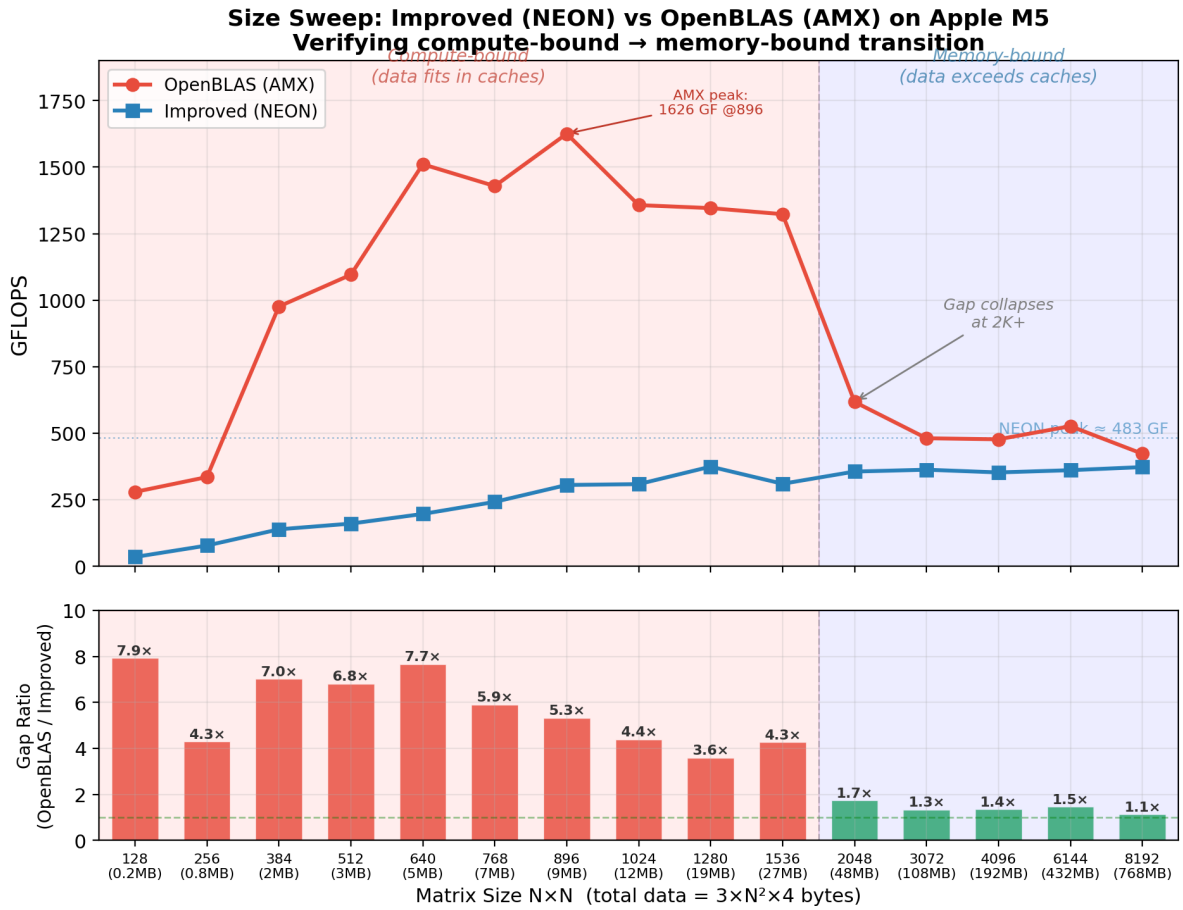

Figure 10: Size sweep confirming the compute-bound → memory-bound transition. Top: absolute GFLOPS. Bottom: gap ratio (OpenBLAS/Improved).

Three distinct regimes are visible:

1. **Compute-bound** ($N \leq 1536$, data ≤ 27 MB): Data fits in L2 caches. AMX runs at full throughput, peaking at **1626 GFLOPS** (N=896). The gap is **4–8**×, reflecting the raw hardware advantage of AMX over NEON.

2. **Transition** (N=2048, data = 48 MB): Data exceeds total L2 capacity (4×4 MB = 16 MB per P-core cluster). OpenBLAS drops from 1323 to 618 GFLOPS; gap collapses to **1.7**×.
3. **Memory-bound** ($N \geq 3072$, data $\geq$ 108 MB): Both implementations are limited by DRAM bandwidth ($\sim$100 GB/s). The gap stabilizes at **1.1**–**1.5**×. Our NEON implementation stays flat at $\sim$360 GFLOPS, demonstrating that our BLIS tiling effectively saturates the available memory bandwidth.

This experiment confirms that our 25% gap at large sizes is *not* a software inefficiency but a *hardware ceiling*—AMX provides only marginal benefit when the memory subsystem is the bottleneck.

5 Conclusion

	Plain	Improved (10T)	Speedup
1024^2	2.0	316.6	158×
8192^2	0.69	385.5	**558**×
65536^2	—	964.3 (32T cloud)	—

Reaching **75% of OpenBLAS** without AMX hardware. The 25% gap is attributable to AMX's dedicated datapath and narrows at larger sizes as both implementations become memory-bandwidth-bound.

A Raw Benchmark Data

A.1 Phase 5: Thread Scaling

```
16x16:    matmul_improved :  0.0000 sec |    4.096 GFLOPS
128x128:  matmul_improved :  0.0002 sec |   22.672 GFLOPS
1024x1024: matmul_improved :  0.0365 sec |   58.908 GFLOPS
8192x8192: matmul_improved : 23.1167 sec |   47.564 GFLOPS
```

Listing 3: 1 thread

```
128x128:  matmul_improved :  0.0001 sec |   36.158 GFLOPS
1024x1024: matmul_improved :  0.0112 sec |  191.381 GFLOPS
8192x8192: matmul_improved :  6.2363 sec |  176.309 GFLOPS
```

Listing 4: 4 threads (P-cores only)

```
128x128:  matmul_improved :  0.0001 sec |   31.775 GFLOPS
1024x1024: matmul_improved :  0.0079 sec |  272.904 GFLOPS
8192x8192: matmul_improved :  3.7576 sec |  292.608 GFLOPS
```

Listing 5: 10 threads (all cores)

A.2 Final Benchmark (10 threads, with OpenBLAS)

```
Matrix size: 16 x 16
matmul_plain         :     0.0000 sec  |     2.048 GFLOPS
matmul_improved      :     0.0003 sec  |     0.029 GFLOPS
OpenBLAS sgemm       :     0.0000 sec  |     1.170 GFLOPS
improved vs plain: match (tol = 1e-04)
improved vs OpenBLAS: match (tol = 1e-03)

```

```
  Matrix size: 128 x 128
  matmul_plain         :     0.0022 sec  |     1.875 GFLOPS
  matmul_improved      :     0.0002 sec  |    25.732 GFLOPS
  OpenBLAS sgemm       :     0.0000 sec  |   279.620 GFLOPS
  improved vs plain: match (tol = 1e-04)
  improved vs OpenBLAS: match (tol = 1e-03)

  Matrix size: 1024 x 1024
  matmul_plain         :     1.0866 sec  |     1.976 GFLOPS
  matmul_improved      :     0.0068 sec  |   316.645 GFLOPS
  OpenBLAS sgemm       :     0.0017 sec  |  1243.476 GFLOPS
  improved vs plain: match (tol = 1e-04)
  improved vs OpenBLAS: match (tol = 1e-03)

  Matrix size: 8192 x 8192
  matmul_improved      :     2.8521 sec  |   385.506 GFLOPS
  OpenBLAS sgemm       :     2.1380 sec  |   514.260 GFLOPS
  improved vs OpenBLAS: match (tol = 1e-03)
```

Listing 6: Final benchmark output

A.3 Plain 8192×8192

```
matmul_plain 8192x8192: 1587.34 sec | 0.693 GFLOPS
```

Listing 7: Plain baseline at 8K

A.4 64K×64K (Cloud)

```
=== 64Kx64K Matrix Multiplication Benchmark ===

Allocating 3 matrices: 51.5 GB total
Allocation OK.

Running matmul_improved...
  matmul_improved :     583.78 sec  |   964.314 GFLOPS

Running OpenBLAS cblas_sgemm...
  OpenBLAS sgemm  :     417.33 sec  |  1348.919 GFLOPS

Correctness check (1000 random samples):
  All samples match (max relative error = 1.34e-05)

=== Summary ===
  Matrix size     : 65536 x 65536
  improved        : 583.78 sec -> 964.314 GFLOPS
  OpenBLAS        : 417.33 sec -> 1348.919 GFLOPS
  improved/BLAS   : 71.5%
```

Listing 8: 64K benchmark (Alibaba Cloud g8y.8xlarge, 32-core ARM)

A.5 Compilation Commands

```
gcc -O3 -Xpreprocessor -fopenmp \
    -I$(brew --prefix libomp)/include \
    -L$(brew --prefix libomp)/lib -lomp \
    -I$(brew --prefix openblas)/include \
```

```
    -L$(brew --prefix openblas)/lib -lopenblas \
    -o benchmark main.c matrix.c matmul_plain.c matmul_improved.c -lm
```

Listing 9: Local (macOS)

```
gcc -O3 -fopenmp -o test_64k \
    test_64k.c matrix.c matmul_plain.c matmul_improved.c \
    -lopenblas -lm
```

Listing 10: Cloud (Linux ARM)